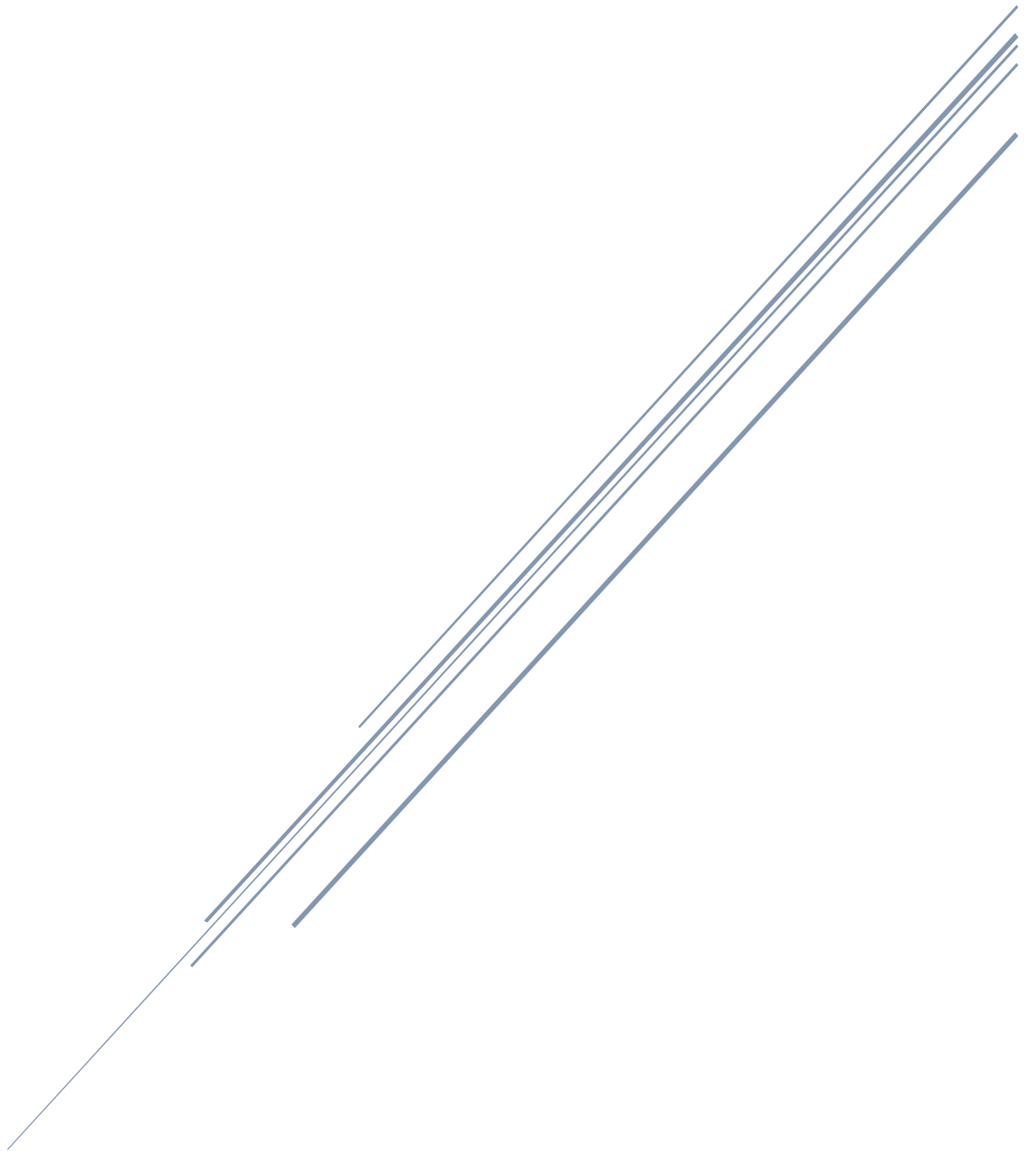


Retail Analytics: Predicting Customer Purchase Behaviour

Capstone Project Report



KAPIL KUMAR MANDAWARIA

Retail Analytics: Predicting Customer Purchase Behaviour

Summary

In the competitive landscape of modern commerce, RetailVC aims to leverage data-driven strategies to gain a competitive edge. This project focuses on analyzing complex retail transaction data to decode customer behavior, demographic influences, and purchasing preferences. By identifying the underlying patterns in how customers interact with various product categories and marketing efforts, the project provides a framework for predicting future responses and optimizing promotional ROI.

Problem Statement:

The goal of this capstone project is to develop a comprehensive understanding of customer behavior and preferences based on retail transaction data. The objective is to develop a comprehensive understanding of the customer base by analyzing:

- **Demographic Profiles:** Age, education, and marital status.
- **Purchasing Habits:** Spending across categories like Wine, Fruits, Meat, and Gold.
- **Campaign Efficacy:** Historical response rates to previous marketing initiatives (Cmp1 through Cmp5).

The goal is to build a predictive model that identifies which customers are most likely to respond to future campaigns, thereby minimizing marketing waste and maximizing conversion rates.

Project Scope & Objectives

The scope of this project covers the entire data lifecycle—from initial cleaning in Excel and SQL to advanced Exploratory Data Analysis (EDA) in Python and executive-level visualization in Tableau.

- **Data Integration & Cleaning:** Consolidating customer data, handling missing income values, and calculating derived metrics like **Customer Age** and **Seniority**.
- **Exploratory Data Analysis (EDA):** Performing univariate and bivariate analysis to identify correlations—such as the relationship between income levels and premium product spending (e.g., Wine).
- **Statistical Anomaly Detection:** Using box plots to identify outliers in income and spending that could skew marketing strategies.
- **Trend Analysis:** Evaluating enrollment trends by year to understand customer acquisition growth.
- **Visualization & Reporting:** Creating an interactive Tableau dashboard to visualize customer distribution and purchase frequency across categories.

The specific goals are to:

1. **Segment the Customer Base:** Categorize customers based on their purchasing power and loyalty.
2. **Predict Response Rates:** Build a foundation for a model that predicts whether a customer will accept a promotional offer (**AcceptedCmp**).

3. **Optimize ROI:** Provide the marketing department with clear evidence of which channels (Web, Catalog, or Store) yield the highest return for specific demographics.

Technical Implementation

Excel Tasks:

Data Exploration:

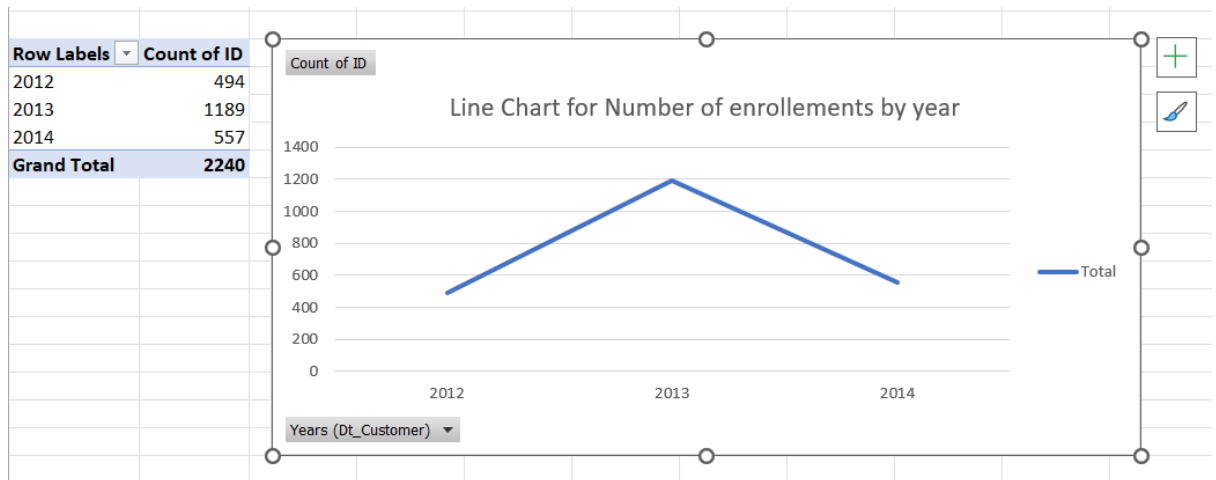
Task 1: Create a statistical summary for numerical features

Income		MntMeatProducts		MntFishProducts	
Mean	52247.25	Mean	166.95	Mean	37.52545
Standard Error	534.7508	Standard Error	4.769108	Standard Error	1.154248
Median	51381.5	Median	67	Median	12
Mode	7500	Mode	7	Mode	0
Standard Deviation	25173.08	Standard Deviation	225.7154	Standard Deviation	54.62898
Sample Variance	6.34E+08	Sample Variance	50947.43	Sample Variance	2984.325
Kurtosis	159.6367	Kurtosis	5.516724	Kurtosis	3.096461
Skewness	6.763487	Skewness	2.083233	Skewness	1.919769
Range	664936	Range	1725	Range	259
Minimum	1730	Minimum	0	Minimum	0
Maximum	666666	Maximum	1725	Maximum	259
Sum	1.16E+08	Sum	373968	Sum	84057
Count	2216	Count	2240	Count	2240

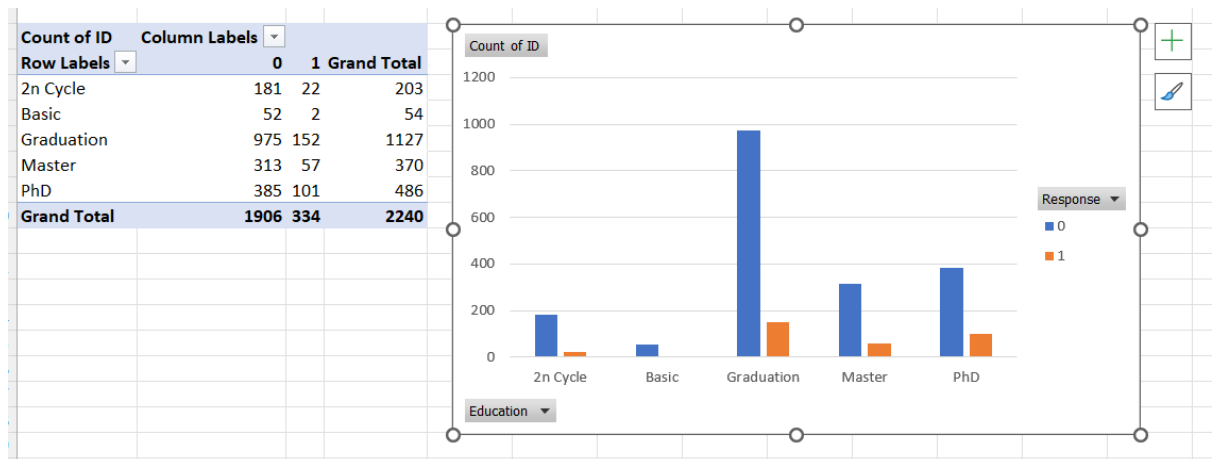
Recency		MntWines		MntFruits	
Mean	49.10938	Mean	303.9357	Mean	26.30223214
Standard Error	0.611944	Standard Error	7.111918	Standard Error	0.840367169
Median	49	Median	173.5	Median	8
Mode	56	Mode	2	Mode	0
Standard Deviation	28.96245	Standard Deviation	336.5974	Standard Deviation	39.77343376
Sample Variance	838.8237	Sample Variance	113297.8	Sample Variance	1581.926033
Kurtosis	-1.2019	Kurtosis	0.598744	Kurtosis	4.050976251
Skewness	-0.00199	Skewness	1.175771	Skewness	2.102063305
Range	99	Range	1493	Range	199
Minimum	0	Minimum	0	Minimum	0
Maximum	99	Maximum	1493	Maximum	199
Sum	110005	Sum	680816	Sum	58917
Count	2240	Count	2240	Count	2240

MntSweetProducts		MntGoldProds		NumDealsPurchases		NumWebPurchases		NumCatalogPurchases		NumStorePurchases		NumWebVisitsMonth	
Mean	27.06295	Mean	44.02188	Mean	2.325	Mean	4.084821	Mean	2.662054	Mean	5.790178571	Mean	5.316517857
Standard Error	0.87221	Standard Error	1.102238	Standard Error	0.040826	Standard Error	0.058711	Standard Error	0.061762	Standard Error	0.068689028	Standard Error	0.051272234
Median	8	Median	24	Median	2	Median	2	Median	2	Median	5	Median	6
Mode	0	Mode	1	Mode	1	Mode	4	Mode	0	Mode	3	Mode	7
Standard Deviation	41.2805	Standard Deviation	52.16744	Standard Deviation	1.932238	Standard Deviation	2.778714	Standard Deviation	2.923101	Standard Deviation	3.250958146	Standard Deviation	2.42664501
Sample Variance	1704.08	Sample Variance	2721.442	Sample Variance	3.733542	Sample Variance	7.721252	Sample Variance	8.544517	Sample Variance	10.56872886	Sample Variance	5.888606002
Kurtosis	4.376548	Kurtosis	3.551709	Kurtosis	8.936914	Kurtosis	5.703128	Kurtosis	8.047437	Kurtosis	-0.62204828	Kurtosis	1.821613827
Skewness	2.136081	Skewness	1.886106	Skewness	2.418569	Skewness	1.382794	Skewness	1.880989	Skewness	0.702237286	Skewness	0.207925557
Range	263	Range	362	Range	15	Range	27	Range	28	Range	13	Range	20
Minimum	0	Minimum	0	Minimum	0	Minimum	0	Minimum	0	Minimum	0	Minimum	0
Maximum	263	Maximum	362	Maximum	15	Maximum	27	Maximum	28	Maximum	13	Maximum	20
Sum	60621	Sum	98609	Sum	5208	Sum	9150	Sum	5963	Sum	12970	Sum	11909
Count	2240	Count	2240	Count	2240	Count	2240	Count	2240	Count	2240	Count	2240

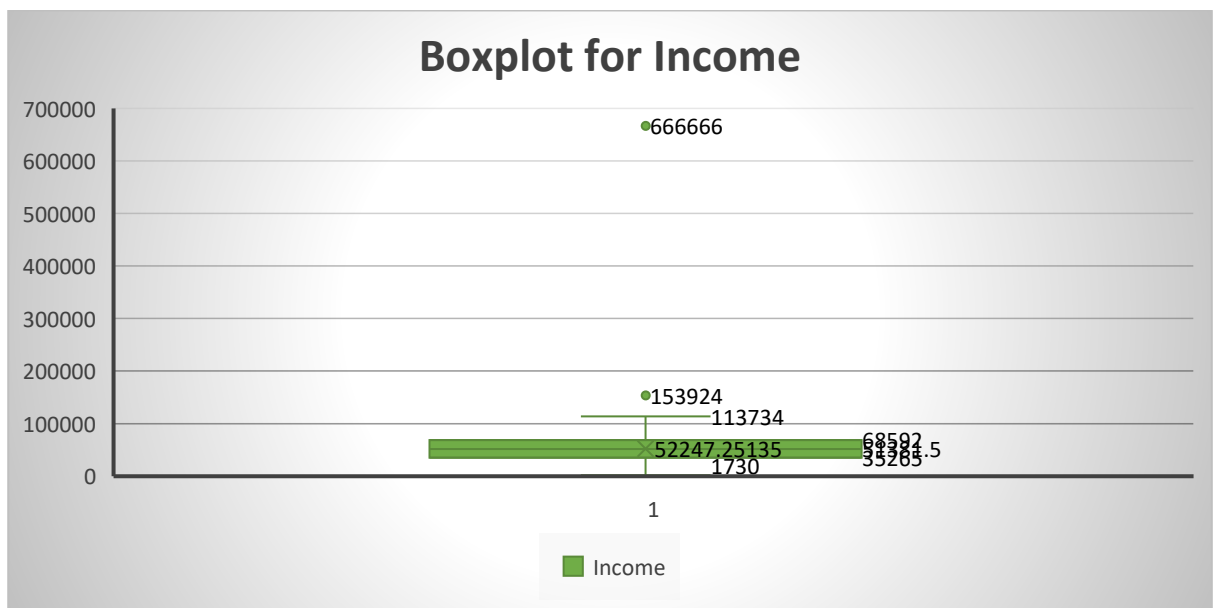
Task 2: Create a line chart for the number of enrolments by year

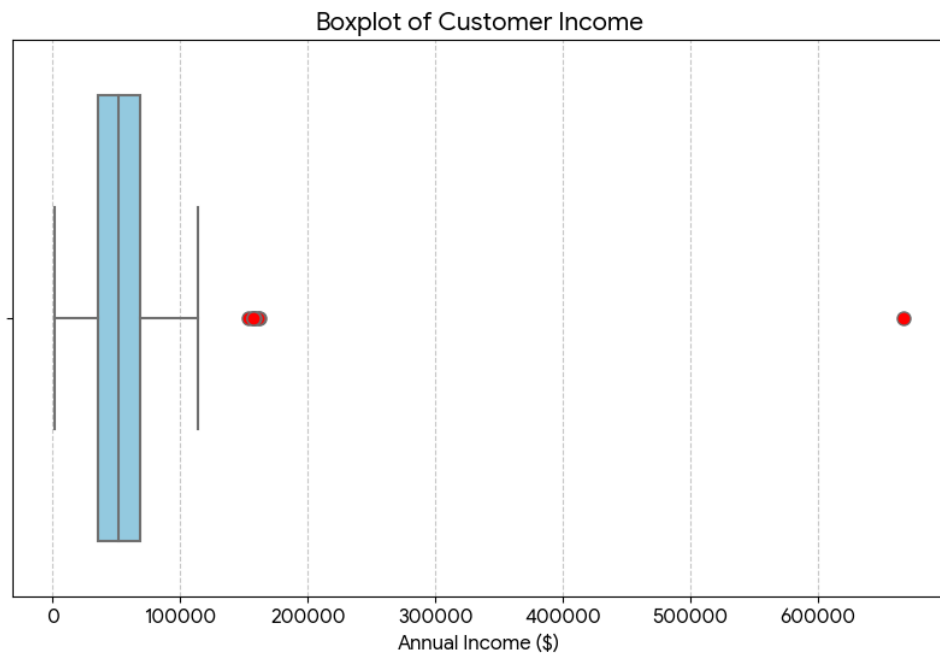


Task 3: Give a cross-tabulated count for response values against education



Task 4: Make a boxplot on income and write your observations

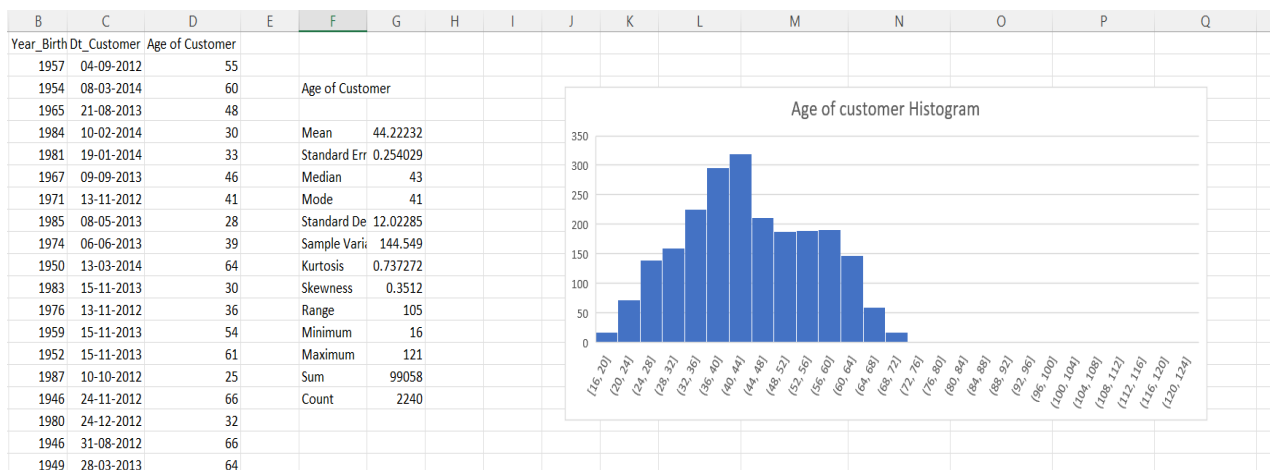




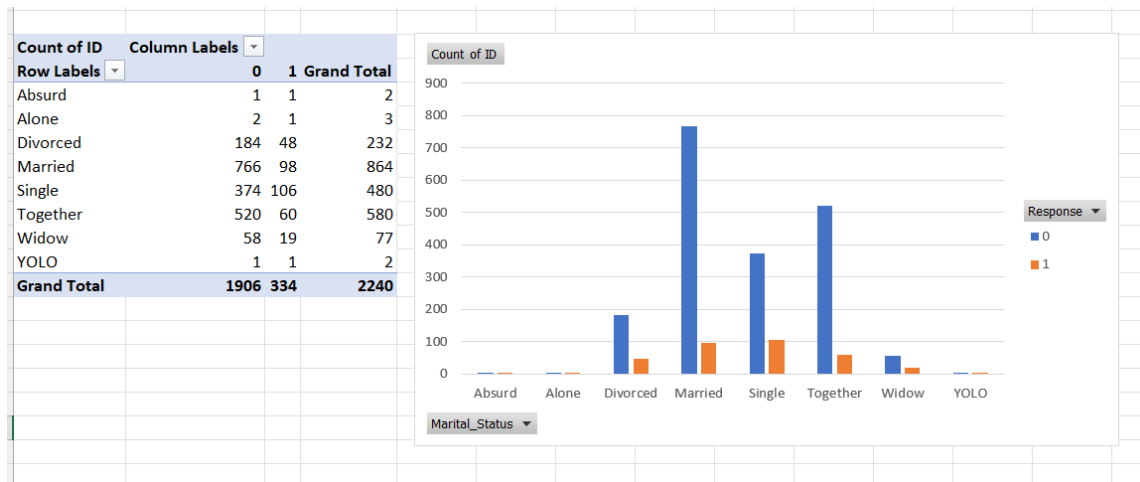
Descriptive Statistics & Observations:

- Extreme Outliers Detected:** The maximum income recorded is \$ 666,666, which is significantly higher than the average. Statistically, any value above \$ 118,350 (the Upper Whisker) is considered an outlier in this dataset. There are 8 such outliers shown as red dots on the plot.
- Central Tendency:** The Median Income is \$ 51,381.50, while the Mean is slightly higher at \$ 52,247.25. This slight difference indicates that the distribution is relatively balanced but has a "right-tail" (positive skew) due to the extreme high earners.
- Middle 50% (IQR):** The majority of your customers (the middle 50%) earn between \$35,303 (25%pct) and \$68,522 (75%pct). This "box" range represents core demographic.
- Spread and Volatility:** The Standard Deviation (\$\sigma\$) is \$25,173, indicating a moderate spread in income levels across the customer base.

Task 5: Calculate the age of customers and make a histogram of that



Task 6: Visualize the response against Marital_Status



SQL Tasks:

Data Loading:

- Create a schema named "retail_data"
- Set "retail_data" as the default schema.

```

1 # Create a schema named "retail_data"
2
3 • CREATE SCHEMA retail_data;
4
5 # Set "retail_data" as the default schema
6
7 • USE retail_data;
8

```

- Create tables to store the retail transaction data
- Set **Dt_customer** as the datetime field while loading the data and apply the appropriate date format

The dataset was successfully created and imported into MySQL using the **Table Data Import Wizard**. This process was initiated by right-clicking on the newly created schema named retail_data, after which the table data was loaded into the database environment for further analysis

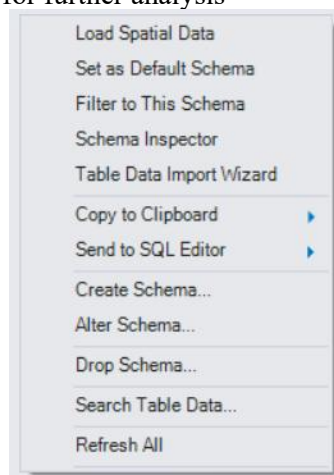


Table Data Import

Configure Import Settings

Detected file format: csv

Encoding: utf-8

Columns:

- ☒ Year_Birth int
- ☒ Education bigint
- ☒ Marital_Status text
- ☒ Income int
- ☒ Kidhome int
- ☒ Teenhome int

ID	Year_Birth	Education	Marital_St...	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWin
5524	1957	Graduation	Single	58138	0	0	04-09-2012	58	635
2174	1954	Graduation	Single	46344	1	1	08-03-2014	38	11
4141	1965	Graduation	Together	71613	0	0	21-08-2013	26	426
6182	1984	Graduation	Together	26646	1	0	10-02-2014	26	11

Date format: %d-%m-%Y

< Back Next > Cancel

Data Preprocessing:

- a) Calculate the total number of customer encounters in the marketing campaign dataset

```

12
13 # Calculate the total number of customer encounters in the marketing campaign dataset
14
15 • SELECT COUNT(*) AS Total_customer_encounters FROM marketing_campaign;
16
17 # Identify the top 10 most purchased products in the dataset, such as Wines, Meat Products, etc
18

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_customer_encounters
2216

- b) Identify the top 10 most purchased products in the dataset, such as Wines, Meat Products, etc.

17 # Identify the top 10 most purchased products in the dataset, such as Wines, Meat Products, etc.marketing_campaign

18

19 • SELECT product, total_spent

20 FROM (

21 SELECT 'Wines' AS product, SUM(MntWines) AS total_spent FROM marketing_campaign

22 UNION ALL

23 SELECT 'Meat Products', SUM(MntMeatProducts) FROM marketing_campaign

24 UNION ALL

25 SELECT 'Fish Products', SUM(MntFishProducts) FROM marketing_campaign

26 UNION ALL

27 SELECT 'Fruits', SUM(MntFruits) FROM marketing_campaign

28 UNION ALL

29 SELECT 'Sweet Products', SUM(MntSweetProducts) FROM marketing_campaign

30 UNION ALL

31 SELECT 'Gold Products', SUM(MntGoldProds) FROM marketing_campaign

Result Grid

product	total_spent
Wines	676083
Meat Products	370063
Gold Products	97427
Fish Products	83405
Sweet Products	59896
Fruits	58405

c) Find the count of response values

36 # Find the count of response values

37

38 • SELECT Response, COUNT(Response) AS Count_of_Response,

39 ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM marketing_campaign), 2) AS response_percentage

40 FROM marketing_campaign

41 group by Response;

42

43 # Determine the distribution of customers based on their education level and marital status

44

45 • SELECT

46 Education,

Result Grid

Response	Count_of_Response	response_percentage
1	333	15.03
0	1883	84.97

d) Determine the distribution of customers based on their education level and marital status


```

74
43 # Determine the distribution of customers based on their education level and marital status
44
45 • SELECT
46     Education,
47     Marital_Status,
48     COUNT(*) AS customer_count,
49     ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM marketing_campaign), 2) AS percentage_share
50 FROM marketing_campaign
51 GROUP BY Education, Marital_Status
52 ORDER BY customer_count DESC;
53
54 • SELECT

```

Education	Marital_Status	customer_count	percentage_share
Graduation	Married	429	19.36
Graduation	Together	285	12.86
Graduation	Single	246	11.10
PhD	Married	190	8.57
Master	Married	138	6.23
Graduation	Divorced	119	5.37
PhD	Together	116	5.23
Master	Together	107	4.60

e) Identify the average income of customers who participated in the marketing campaign

```

53
54 # Identify the average income of customers who participated in the marketing campaign
55
56 • SELECT AVG(Income) AS Average_Income FROM marketing_campaign;
57
58 • SELECT Response, SUM(Income),
59     AVG(Income) AS avg_income_participants
60 FROM marketing_campaign
61 group by Response;
62
63 # Calculate the total number of promotions accepted by customers in each campaign
64
65 • SELECT
66     SUM(AcceptedCmp1) AS total_cmp1_accepted,
67     SUM(AcceptedCmp2) AS total_cmp2_accepted,

```

Response	SUM(Income)	avg_income_participants
1	20049822	60209.6757
0	95730087	50839.1328

f) Calculate the total number of promotions accepted by customers in each campaign

```

62
63 # Calculate the total number of promotions accepted by customers in each campaign
64
65 • SELECT
66     SUM(AcceptedCmp1) AS total_cmp1_accepted,
67     SUM(AcceptedCmp2) AS total_cmp2_accepted,
68     SUM(AcceptedCmp3) AS total_cmp3_accepted,
69     SUM(AcceptedCmp4) AS total_cmp4_accepted,
70     SUM(AcceptedCmp5) AS total_cmp5_accepted,
71     SUM(Response) AS total_final_campaign_accepted
72 FROM marketing_campaign;
73

```

	total_cmp1_accepted	total_cmp2_accepted	total_cmp3_accepted	total_cmp4_accepted	total_cmp5_accepted	total_final_campaign_accepted
▶	142	30	163	164	162	333

g) Identify the distribution of customers' responses to the last campaign

```

76 # Identify the distribution of customers' responses to the last campaign
77 • SELECT
78     AcceptedCmp5,
79     Response,
80     COUNT(*) AS Customer_Count
81 FROM
82     marketing_campaign
83 GROUP BY
84     AcceptedCmp5,
85     Response
86 ORDER BY
87     AcceptedCmp5,
88     Response;

```

	AcceptedCmp5	Response	Customer_Count
▶	0	0	1812
	0	1	242
	1	0	71
	1	1	91

h) Calculate the average number of children and teenagers in customers' households

```

86 # Calculate the average number of children and teenagers in customers' households
87
88 • SELECT
89     AVG(Kidhome) AS AVG_Children,
90     AVG(Teenhome) AS AVG_Teenagers,
91     AVG(Kidhome + Teenhome) AS avg_dependents
92 FROM marketing_campaign;
93
94 # Create an Age column by subtracting year_birth from the current year
95
96 • SELECT ID,

```

Result Grid	Filter Rows:	Exports:	Wrap Cell Content:
AVG_Children	AVG_Teenagers	avg_dependents	
0.4418	0.5054	0.9472	

i) Create an Age column by subtracting year_birth from the current year

```

106 # Create an Age column by subtracting year_birth from the current year
107
108 • SELECT ID,
109     YEAR(Dt_Customer) - Year_Birth as Age_when_Erolled
110 FROM marketing_campaign;
111
112 • SELECT *,
113     (YEAR(CURDATE()) - Year_Birth) AS Age
114 FROM marketing_campaign;
115

```

Result Grid

Filter Rows:

Export:

Wrap Cell Contents:

Fetch rows:

	NumCatalogPurchases	NumStorePurchases	NumWebVisitsMonth	AcceptedCmp3	AcceptedCmp4	AcceptedCmp5	AcceptedCmp1	AcceptedCmp2	Complain	Response	Age	Age
0	4	7	0	0	0	0	0	0	0	1	55	69
	2	5	0	0	0	0	0	0	0	0	60	72
	10	4	0	0	0	0	0	0	0	0	48	61
	4	6	0	0	0	0	0	0	0	0	30	42
	6	5	0	0	0	0	0	0	0	0	33	45
	10	6	0	0	0	0	0	0	0	0	46	59
	7	6	0	0	0	0	0	0	0	0	41	55

Result 21

j) Create Age_group columns based on the below condition:
 WHEN Age BETWEEN 18 AND 25 THEN '18-25'
 WHEN Age BETWEEN 26 AND 35 THEN '26-35'
 WHEN Age BETWEEN 36 AND 45 THEN '36-45'
 WHEN Age BETWEEN 46 AND 55 THEN '46-55'
 ELSE '56+'

```

126 # Create Age_group columns
127 • SELECT *,
128     CASE
129         WHEN Age BETWEEN 18 AND 25 THEN '18-25'
130         WHEN Age BETWEEN 26 AND 35 THEN '26-35'
131         WHEN Age BETWEEN 36 AND 45 THEN '36-45'
132         WHEN Age BETWEEN 46 AND 55 THEN '46-55'
133         ELSE '56+'
134     END AS Age_group
135 FROM (
136     /* Calculating Age first */
137     SELECT *, (YEAR(CURDATE()) - Year_Birth) AS Age
138     FROM marketing_campaign
139 ) AS customer_data;

```

catalogPurchases	NumStorePurchases	NumWebVisitsMonth	AcceptedCmp3	AcceptedCmp4	AcceptedCmp5	AcceptedCmp1	AcceptedCmp2	Complain	Response	Age	Age_group
4	7	0	0	0	0	0	0	0	1	69	56+
2	5	0	0	0	0	0	0	0	0	72	56+
10	4	0	0	0	0	0	0	0	0	61	56+
4	6	0	0	0	0	0	0	0	0	42	36-45
6	5	0	0	0	0	0	0	0	0	45	36-45
10	6	0	0	0	0	0	0	0	0	59	56+

k) Determine the average number of visits per month for customers in each age group

```

153 # Determine the average number of visits per month for customers in each age group
154 • SELECT
155     Age_group,
156     AVG(NumWebVisitsMonth + NumStorePurchases) AS Avg_Monthly_Visits
157 FROM (
158     SELECT *,
159     CASE
160         WHEN (2024 - Year_Birth) BETWEEN 18 AND 25 THEN '18-25'
161         WHEN (2024 - Year_Birth) BETWEEN 26 AND 35 THEN '26-35'
162         WHEN (2024 - Year_Birth) BETWEEN 36 AND 45 THEN '36-45'
163         WHEN (2024 - Year_Birth) BETWEEN 46 AND 55 THEN '46-55'
164         ELSE '56+'
165     END AS Age_group
166     FROM marketing_campaign
167 ) AS categorized_data

```

Age_group	Avg_Monthly_Visits
56+	11.2837
46-55	11.1173
36-45	10.8037
26-35	10.6778

Python Tasks:

Task 1: General Data Overview:

- ❖ Check the first few rows of the dataset to understand its structure

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as sc
import math

[2]: # Load the dataset
# Replace "your_dataset.csv" with the path to your dataset
df = pd.read_csv(r"C:\Users\kalya\OneDrive\Documents\KAPIL_DATAanalysis\Capstone Project\Retail_Analytics_Datasets\marketing_campaign.csv")

[3]: # Display the first few rows to verify the data was loaded correctly
print("First 5 rows of the data:")
display(df.head())
```

First 5 rows of the data:

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...	NumCatalogPurchases	NumStorePurchases	NumV
0	5524	1957	Graduation	Single	58138.0	0	0	04-09-2012	58	635	...	10	4	
1	2174	1954	Graduation	Single	46344.0	1	1	08-03-2014	38	11	...	1	2	
2	4141	1965	Graduation	Together	71613.0	0	0	21-08-2013	26	426	...	2	10	
3	6182	1984	Graduation	Together	26646.0	1	0	10-02-2014	26	11	...	0	4	
4	5324	1981	PhD	Married	58293.0	1	0	19-01-2014	94	173	...	3	6	

5 rows × 27 columns

❖ Check the data types of each column

```
[4]: # Check the data types of each column
# Get basic information about the dataframe
print("\nDataframe info:")
display(df.info())
```

```
Dataframe info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 27 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   ID                    2240 non-null   int64
 1   Year_Birth            2240 non-null   int64
 2   Education             2240 non-null   object
 3   Marital_Status        2240 non-null   object
 4   Income                2216 non-null   float64
 5   Kidhome               2240 non-null   int64
 6   Teenhome              2240 non-null   int64
 7   Dt_Customer           2240 non-null   object
 8   Recency               2240 non-null   int64
 9   MntWines              2240 non-null   int64
10   MntFruits             2240 non-null   int64
11   MntMeatProducts       2240 non-null   int64
12   MntFishProducts       2240 non-null   int64
13   MntSweetProducts      2240 non-null   int64
14   MntGoldProds          2240 non-null   int64
15   NumDealsPurchases     2240 non-null   int64
16   NumWebPurchases       2240 non-null   int64
17   NumCatalogPurchases   2240 non-null   int64
18   NumStorePurchases     2240 non-null   int64
19   NumWebVisitsMonth     2240 non-null   int64
20   AcceptedCmp3          2240 non-null   int64
21   AcceptedCmp4          2240 non-null   int64
22   AcceptedCmp5          2240 non-null   int64
23   AcceptedCmp1          2240 non-null   int64
24   AcceptedCmp2          2240 non-null   int64
25   Complain              2240 non-null   int64
26   Response              2240 non-null   int64
dtypes: float64(1), int64(23), object(3)
memory usage: 472.6+ KB
None
```

❖ Check for any missing values in the dataset

```

•[8]: # Check for any missing values in the dataset
      # Check for missing values in each column
      missing_values = df.isnull().sum()
      # Display columns with missing values
      print("Missing values per column:")
      print(missing_values)
      # Optionally, show only columns that have missing values
      print("\nColumns with missing values:")
      print(missing_values[missing_values > 0])

```

```

Missing values per column:
ID                0
Year_Birth        0
Education         0
Marital_Status    0
Income           24
Kidhome           0
Teenhome          0
Dt_Customer       0
Recency           0
MntWines          0
MntFruits         0
MntMeatProducts   0
MntFishProducts   0
MntSweetProducts  0
MntGoldProds      0
NumDealsPurchases 0
NumWebPurchases   0
NumCatalogPurchases 0
NumStorePurchases 0
NumWebVisitsMonth 0
AcceptedCmp3      0
AcceptedCmp4      0
AcceptedCmp5      0
AcceptedCmp1      0
AcceptedCmp2      0
Complain          0
Response          0
dtype: int64

Columns with missing values:

```

Task 2: Descriptive Statistics:

- ❖ Compute summary statistics for numerical columns (mean, median, min, max, and standard deviation)

```

[5]: # Compute summary statistics for numerical columns (mean, median, min, max, and standard deviation)
      # Get basic statistics of the numerical columns

      print("\nSummary statistics:")
      display(df.describe())

```

Summary statistics:

	ID	Year_Birth	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	MntFishProducts	...	NumCatalog
count	2240.000000	2240.000000	2216.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	2240.000000	...	2240.000000
mean	5592.159821	1968.805804	52247.251354	0.444196	0.506250	49.109375	303.935714	26.302232	166.950000	37.525446	...	37.525446
std	3246.662198	11.984069	25173.076661	0.538398	0.544538	28.962453	336.597393	39.773434	225.715373	54.628979	...	54.628979
min	0.000000	1893.000000	1730.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000
25%	2828.250000	1959.000000	35303.000000	0.000000	0.000000	24.000000	23.750000	1.000000	16.000000	3.000000	...	3.000000
50%	5458.500000	1970.000000	51381.500000	0.000000	0.000000	49.000000	173.500000	8.000000	67.000000	12.000000	...	12.000000
75%	8427.750000	1977.000000	68522.000000	1.000000	1.000000	74.000000	504.250000	33.000000	232.000000	50.000000	...	50.000000
max	11191.000000	1996.000000	666666.000000	2.000000	2.000000	99.000000	1493.000000	199.000000	1725.000000	259.000000	...	259.000000

8 rows × 24 columns

The maximum income recorded is \$ 666,666, which is significantly higher than the mean, so we will Address missing values in Income column by Median value.

```
[7]: # 2. Cleaning and Feature Engineering
df['Income'] = pd.to_numeric(df['Income'], errors='coerce')
df['Income'] = df['Income'].fillna(df['Income'].median())
```

❖ Explore the distribution of numerical variables using histograms or box plots

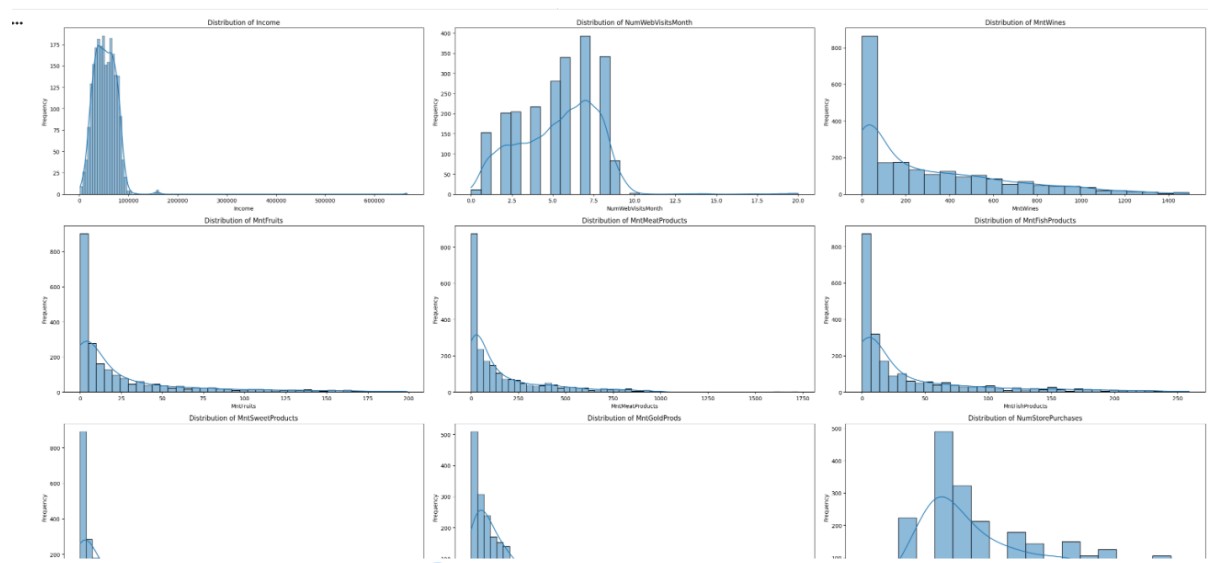
```
[8]: # Explore the distribution of numerical variables using histograms or box plots

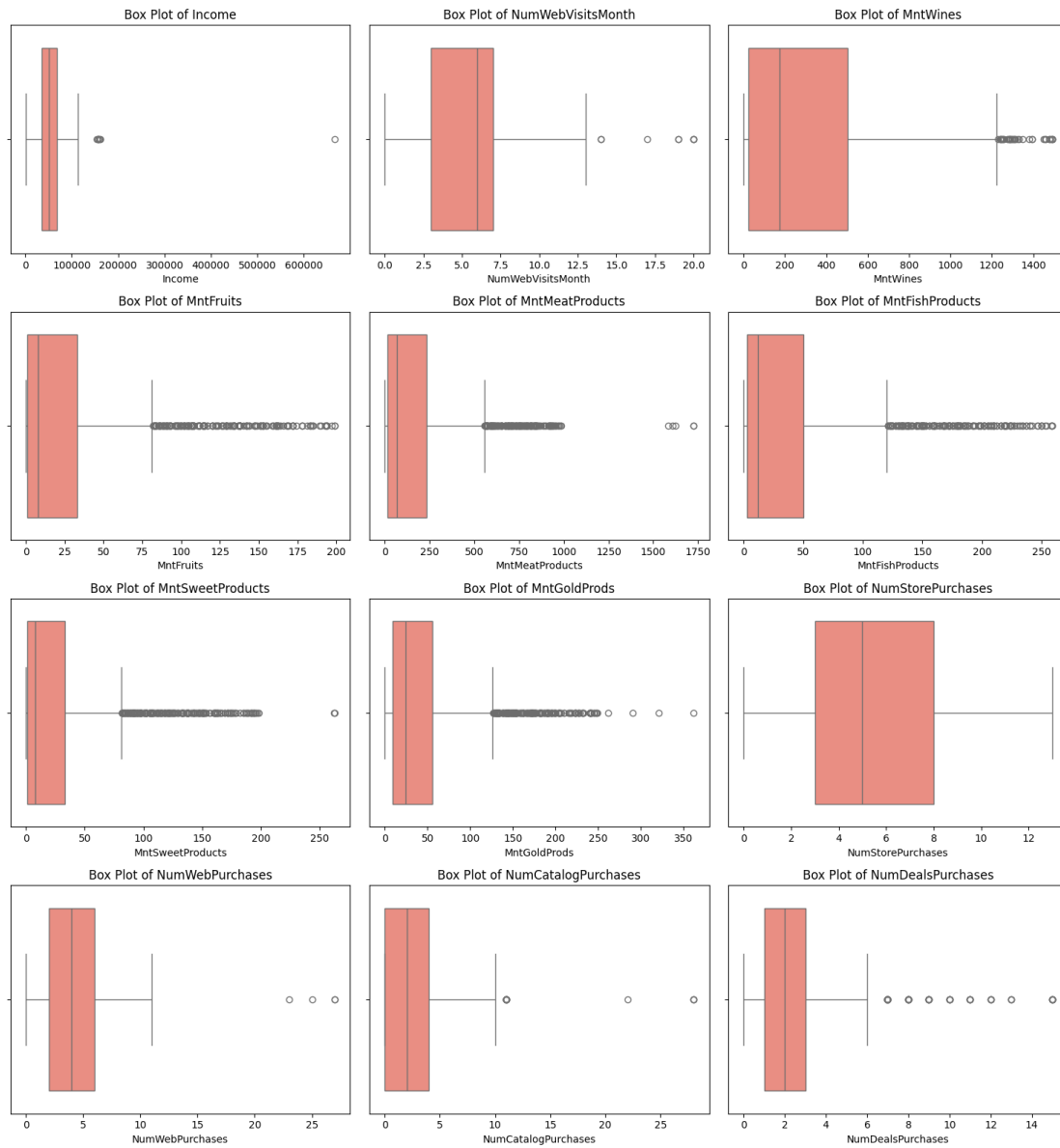
# Select numerical columns of interest
num_cols = ["Income", "NumWebVisitsMonth", "MntWines",
            "MntFruits", "MntMeatProducts", "MntFishProducts", "MntSweetProducts", "MntGoldProds",
            "NumStorePurchases", "NumWebPurchases",
            "NumCatalogPurchases", "NumDealsPurchases"]

# Dynamically calculate grid size (3 columns wide)
n_rows = math.ceil(len(num_cols) / 3) # Removed indentation here

# 1. Histograms
plt.figure(figsize=(30, 30))
# Adjusted grid size to accommodate all 26 numerical columns (e.g., 6 rows * 5 columns = 30 plots)
for i, col in enumerate(num_cols):
    plt.subplot(6, 3, i + 1)
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()
plt.savefig('histograms1.png') # In this environment, use savefig
##

# 2. Box plots
plt.figure(figsize=(15, n_rows * 4)) # Removed indentation here
for i, col in enumerate(num_cols, 1):
    plt.subplot(n_rows, 3, i)
    sns.boxplot(x=df[col].dropna(), color='salmon') # Changed df[num_cols] to df[col]
    plt.title(f'Box Plot of {col}') # Changed num_cols to col
plt.tight_layout()
plt.savefig('boxplots.png')
```





Task 3: Univariate Analysis:

- ❖ Explore the distribution of each numerical variable using histograms or kernel density plots

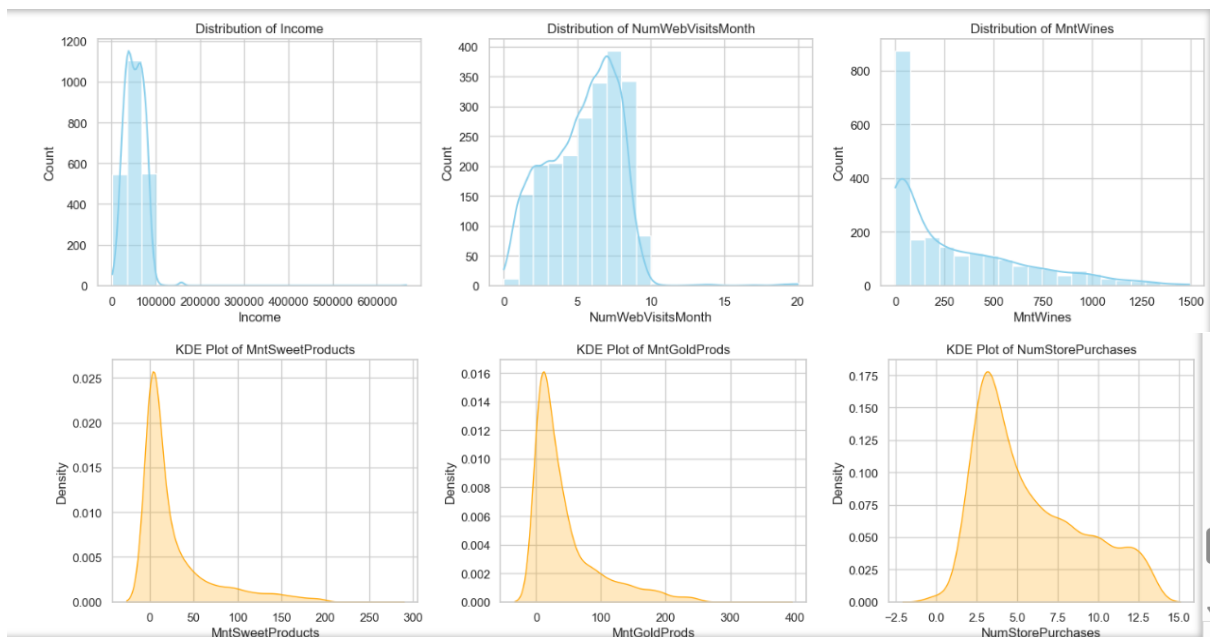

```
[20]: # Task 3: Univariate Analysis
# Explore the distribution of each numerical variable using histograms or kernel density plots

# Dynamically calculate grid size (3 columns wide)
n_rows = math.ceil(len(num_cols) / 3) # Removed indentation here

# Set style
sns.set(style="whitegrid")

# Histograms with KDE overlay
plt.figure(figsize=(15, n_rows * 4))
for i, col in enumerate(num_cols, 1):
    plt.subplot(n_rows, 3, i)
    sns.histplot(df[col].dropna(), kde=True, bins=20, color="skyblue")
    plt.title(f"Distribution of {col}")
plt.tight_layout()
plt.show()

# Kernel Density Plots separately
plt.figure(figsize=(15, n_rows * 4))
for i, col in enumerate(num_cols, 1):
    plt.subplot(n_rows, 3, i)
    sns.kdeplot(df[col].dropna(), fill=True, color="orange")
    plt.title(f"KDE Plot of {col}")
plt.tight_layout()
plt.show()
```



❖ Explore the distribution of each categorical variable using bar plots or pie charts

```
# Explore the distribution of each categorical variable using bar plots or pie charts
```

```
# 1. Feature Engineering: Dates and Age
```

```
# Using dayfirst=True to handle the 'DD-MM-YYYY' format correctly
```

```
df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'], dayfirst=True)
```

```
# 2. Calculate Age
```

```
current_year = pd.Timestamp.now().year # Changed from pd.to_datetime.now() to pd.Timestamp.now()
```

```
df['Age'] = current_year - df['Year_Birth']
```

```
# 3. Define bins and Labels
```

```
bins = [18, 25, 35, 45, 55, 120] # 120 is an upper bound for '56+'
```

```
labels = ['18-25', '26-35', '36-45', '46-55', '56+']
```

```
# 4. Create Age_group column
```

```
df['Age_group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=True)
```

```
print(df[['Age', 'Age_group']].head())
```

```
# 5. Define categorical columns for plotting
```

```
cat_cols = ['Education', 'Marital_Status', 'Age_group', 'Kidhome', 'Teenhome']
```

```
# 6. Generate Bar Plots
```

```
plt.figure(figsize=(18, 10))
```

```
for i, col in enumerate(cat_cols, 1):
```

```
    plt.subplot(2, 3, i)
```

```
    # Order bars by frequency
```

```
    sns.countplot(data=df, x=col, order=df[col].value_counts().index, palette="Set2")
```

```
    plt.title(f"Distribution of {col}", fontsize=14)
```

```
    plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.savefig('categorical_bar_plots.png')
```

```
# 7. Generate Pie Charts
```

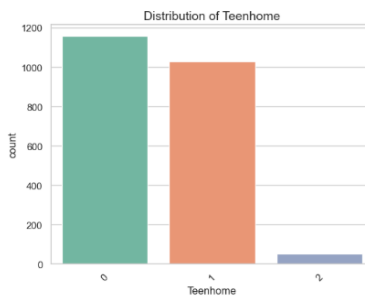
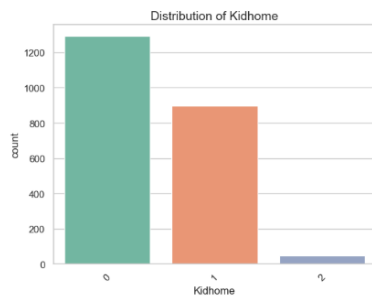
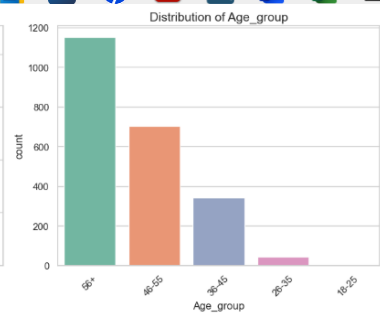
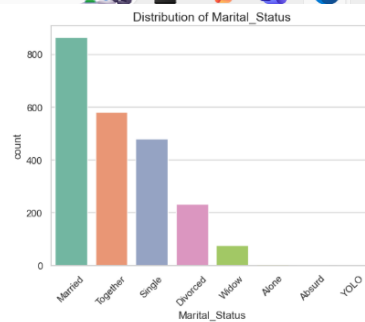
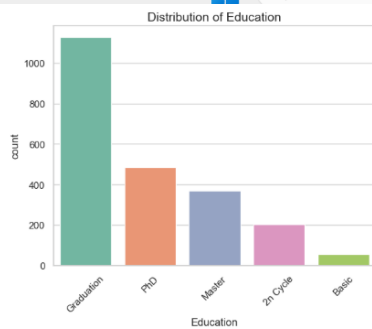
```
plt.figure(figsize=(18, 10))
```

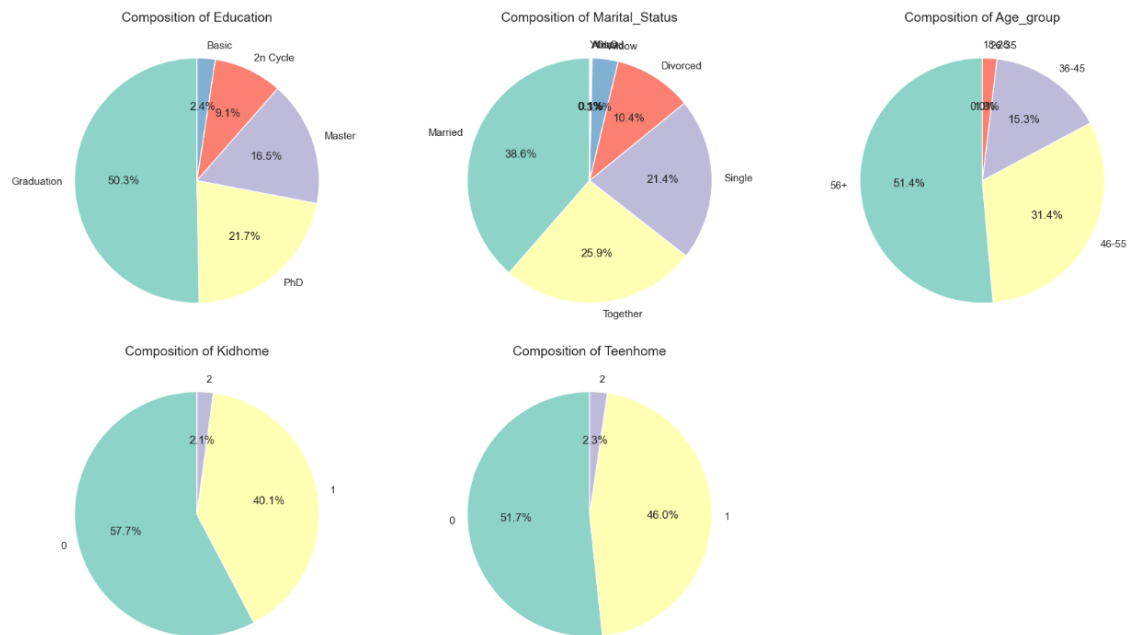
```
for i, col in enumerate(cat_cols, 1):
```

```
    plt.subplot(2, 3, i)
```

```
    counts = df[col].value_counts()
```

```
    plt.pie(counts, labels=counts.index, autopct='%1.1f%%', startangle=90, colors=sns.color_palette("Set3"))
```





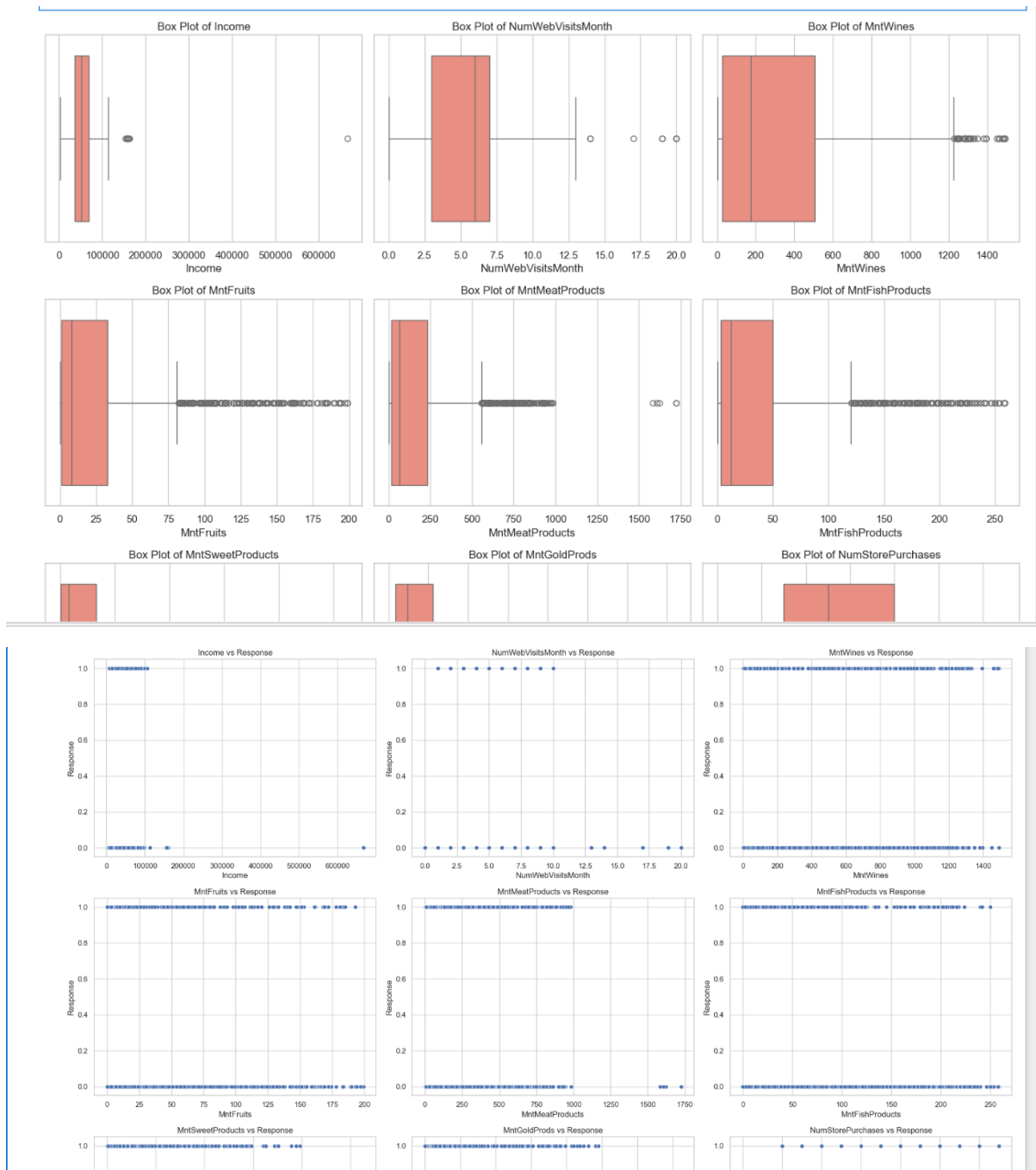
❖ Identify outliers in numerical variables using box plots or scatter plots

```
# Select numerical columns of interest
num_cols = ["Income", "NumWebVisitsMonth", "MntWines",
            "MntFruits", "MntMeatProducts", "MntFishProducts", "MntSweetProducts", "MntGoldProds",
            "NumStorePurchases", "NumWebPurchases",
            "NumCatalogPurchases", "NumDealsPurchases"]

# Dynamically calculate grid size (3 columns wide)
n_rows = math.ceil(len(num_cols) / 3)
# 2. Box plots
plt.figure(figsize=(15, n_rows * 4))
for i, col in enumerate(num_cols, 1):
    plt.subplot(n_rows, 3, i)
    sns.boxplot(x=df[col].dropna(), color='salmon')
    plt.title(f"Box Plot of {col}")
plt.tight_layout()
plt.savefig('boxplots.png')

# Scatter Plots
# Exclude 'Response' from the list if it's already in numerical_cols to plot against it
if 'Response' in num_cols:
    num_cols.remove('Response')

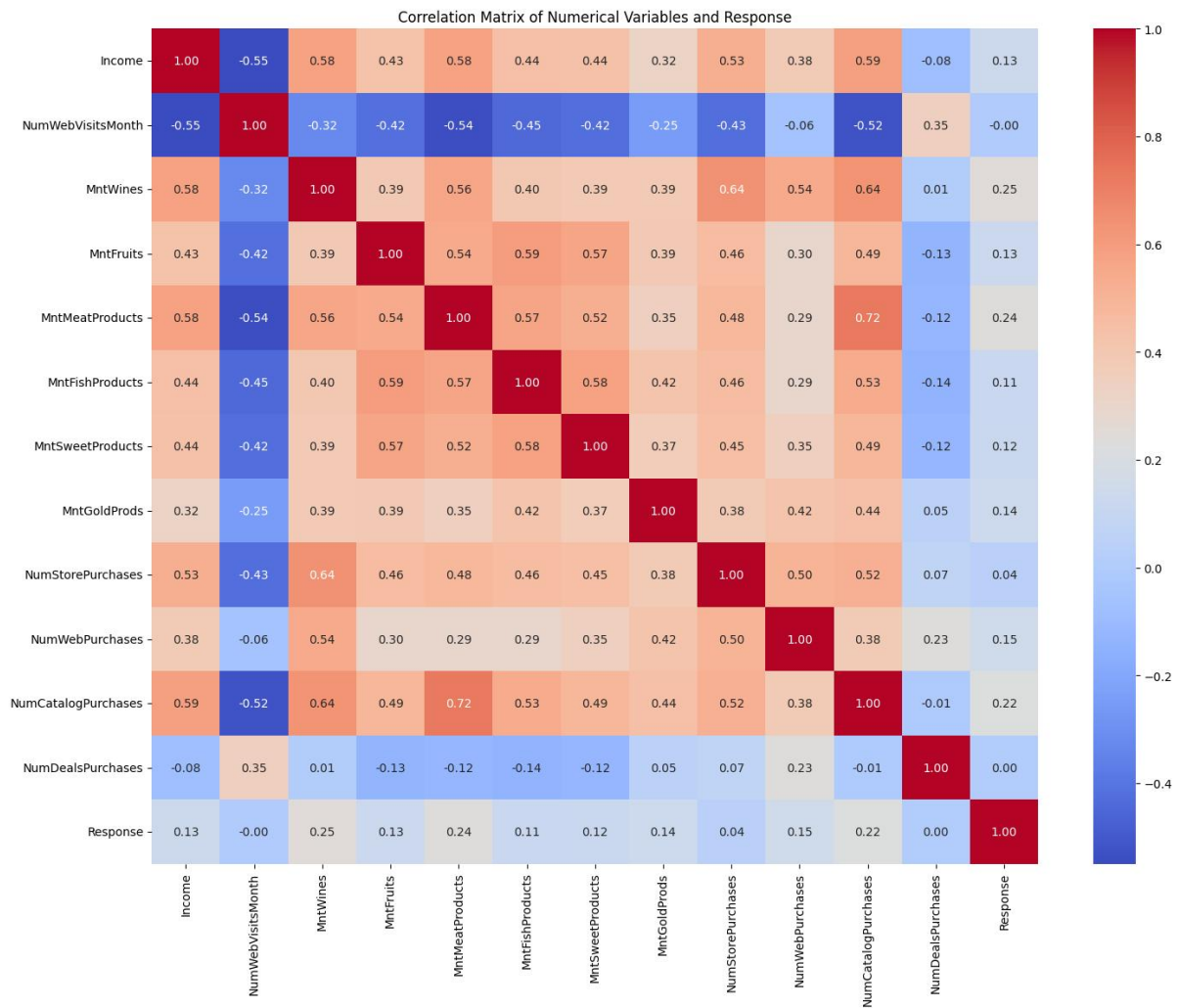
# Scatter plots of numerical features vs Response
plt.figure(figsize=(20, 20))
n_cols_per_row = 3
n_rows = math.ceil(len(num_cols) / n_cols_per_row)
for i, col in enumerate(num_cols):
    plt.subplot(n_rows, n_cols_per_row, i + 1)
    sns.scatterplot(x=df[col], y=df['Response'])
    plt.title(f'{col} vs Response')
    plt.xlabel(col)
    plt.ylabel('Response')
plt.tight_layout()
plt.savefig('numerical_vs_response_scatter.png')
plt.show()
```



Task 4: Bivariate Analysis:

- ❖ Explore the relationship between numerical variables and the target variable (**Response**) using scatter plots or correlation matrices

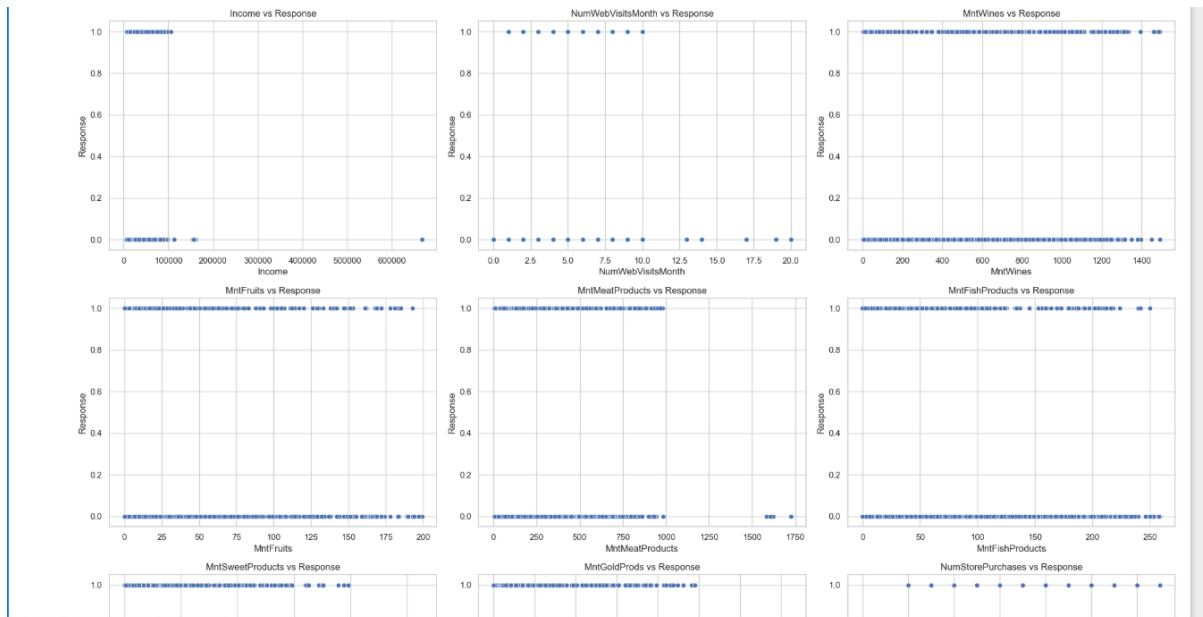
```
# Correlation Matrix
plt.figure(figsize=(15, 12))
sns.heatmap(df[numerical_cols + ['Response']].corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Variables and Response')
plt.tight_layout()
plt.savefig('correlation_matrix.png')
plt.show()
```



```
numerical_cols = ["Income", "NumWebVisitsMonth", "MntWines",
                  "MntFruits", "MntMeatProducts", "MntFishProducts", "MntSweetProducts", "MntGoldProds",
                  "NumStorePurchases", "NumWebPurchases",
                  "NumCatalogPurchases", "NumDealsPurchases"]
# Exclude 'Response' from the list if it's already in numerical_cols to plot against it
if 'Response' in numerical_cols:
    numerical_cols.remove('Response')

# Scatter plots of numerical features vs Response
plt.figure(figsize=(20, 20))
n_cols_per_row = 3
n_rows = math.ceil(len(numerical_cols) / n_cols_per_row)

for i, col in enumerate(numerical_cols):
    plt.subplot(n_rows, n_cols_per_row, i + 1)
    sns.scatterplot(x=df[col], y=df['Response'])
    plt.title(f'{col} vs Response')
    plt.xlabel(col)
    plt.ylabel('Response')
plt.tight_layout()
plt.savefig('numerical_vs_response_scatter.png')
plt.show()
```

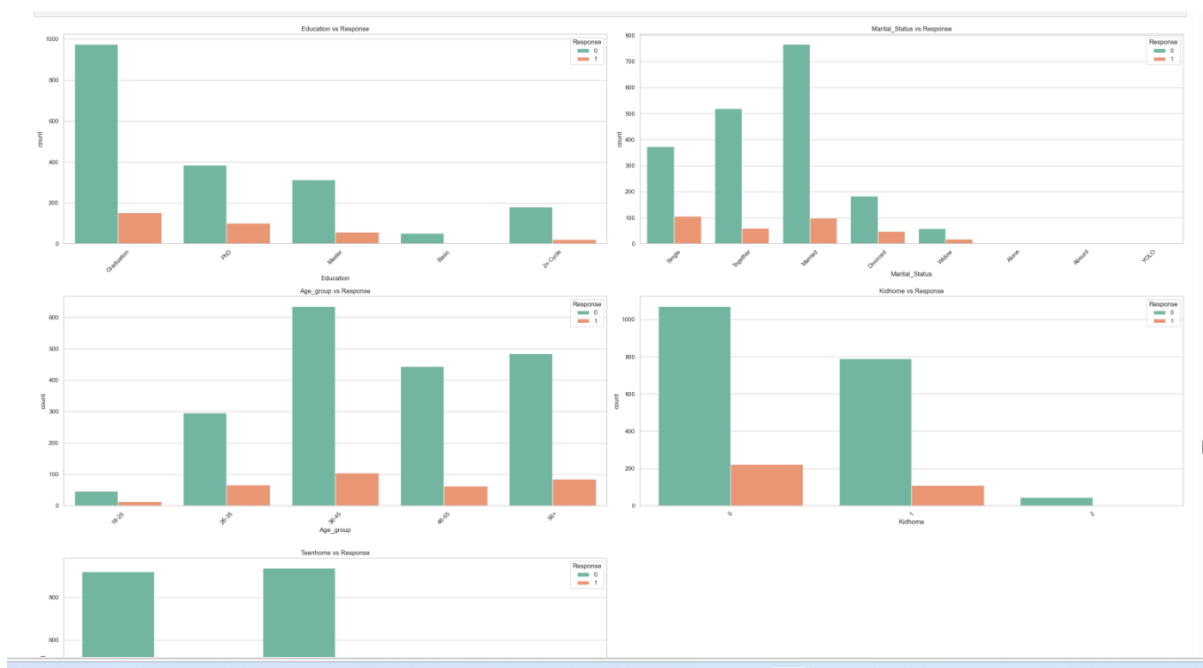


❖ Explore the relationship between categorical variables and the target variable using bar plots or chi-square tests

```
categorical_cols = ['Education', 'Marital_Status', 'Age_group', 'Kidhome', 'Teenhome'] # Exclude Response and Complain for this analysis as Response is target and Complain is a b

for col in categorical_cols:
    plt.figure(figsize=(8, 5))
    sns.countplot(data=df, x=col, hue='Response', palette='viridis')
    plt.title(f'Response Rate by {col}')
    plt.xlabel(col)
    plt.ylabel('Count')
    plt.xticks(rotation=45, ha='right')
    plt.legend(title='Response')
    plt.tight_layout()
    plt.show()

    print(f"\nChi-square test for {col} vs Response:")
    contingency_table = pd.crosstab(df[col], df['Response'])
    chi2, p_value, _, _ = sc.chi2_contingency(contingency_table)
    print(f"Chi-square statistic: {chi2:.2f}")
    print(f"P-value: {p_value:.3f}")
    if p_value < 0.05:
        print(f"There is a significant relationship between {col} and Response (p < 0.05).")
    else:
        print(f"There is no significant relationship between {col} and Response (p >= 0.05).")
```



Chi-square test for Education vs Response:

Chi-square statistic: 23.08

P-value: 0.0001

There is a significant relationship between Education and Response ($p < 0.05$).

Chi-square test for Marital_Status vs Response:

Chi-square statistic: 54.24

P-value: 0.0000

There is a significant relationship between Marital_Status and Response ($p < 0.05$).

Chi-square test for Age_group vs Response:

Chi-square statistic: 9.42

P-value: 0.0514

There is no significant relationship between Age_group and Response ($p \geq 0.05$).

Chi-square test for Kidhome vs Response:

Chi-square statistic: 14.64

P-value: 0.0007

There is a significant relationship between Kidhome and Response ($p < 0.05$).

Chi-square test for Teenhome vs Response:

Chi-square statistic: 58.34

P-value: 0.0000

There is a significant relationship between Teenhome and Response ($p < 0.05$).

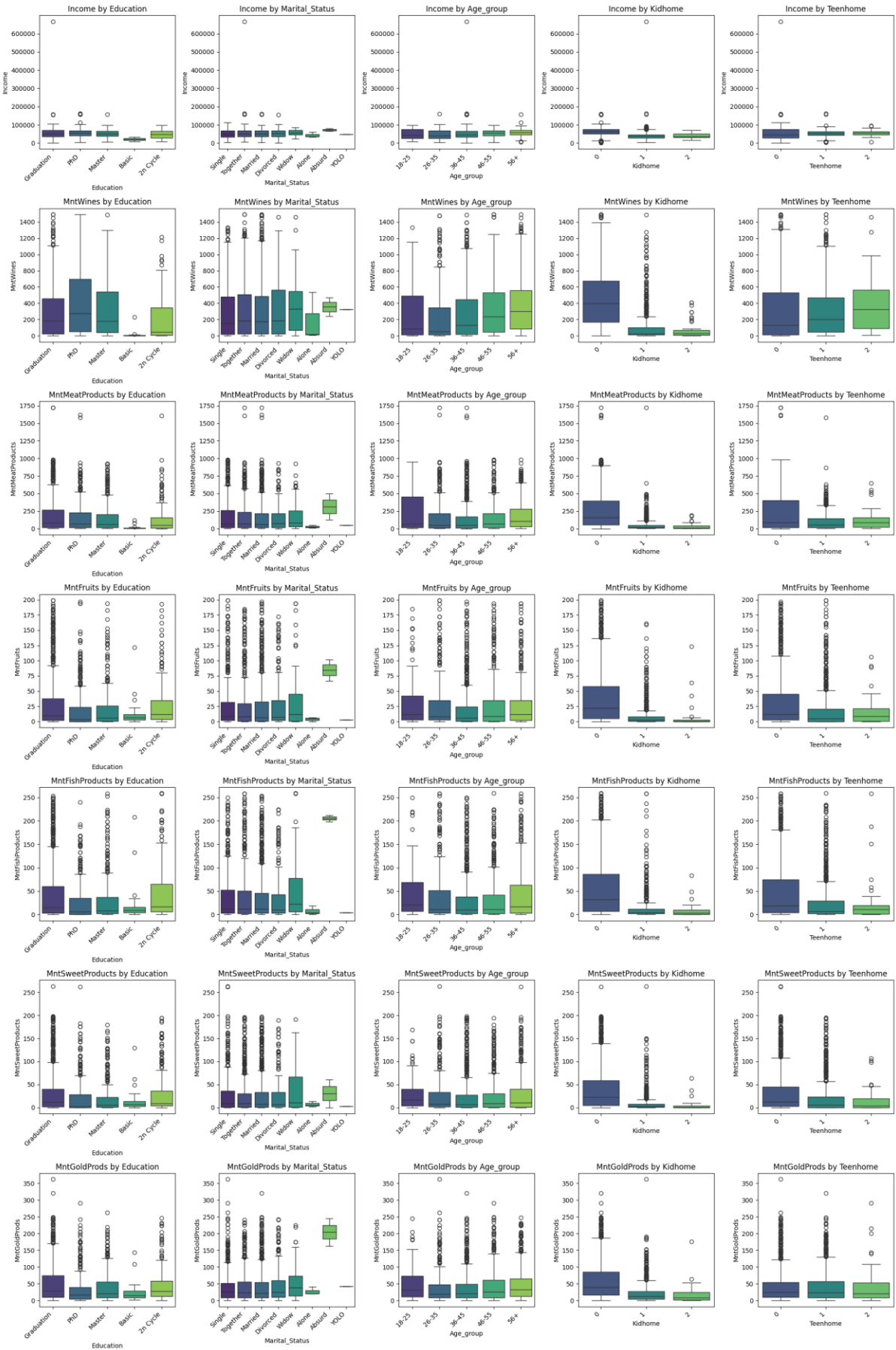
- ❖ Explore the relationship between numerical and categorical variables using box plots or violin plots

```
# Define numerical and categorical columns for this analysis
selected_numerical_cols = ['Income', 'MntWines', 'MntMeatProducts', 'MntFruits', 'MntFishProducts', 'MntSwe']
selected_categorical_cols = ['Education', 'Marital_Status', 'Age_group', 'Kidhome', 'Teenhome']

# Convert Kidhome and Teenhome to category type for proper plotting
df['Kidhome'] = df['Kidhome'].astype('category')
df['Teenhome'] = df['Teenhome'].astype('category')

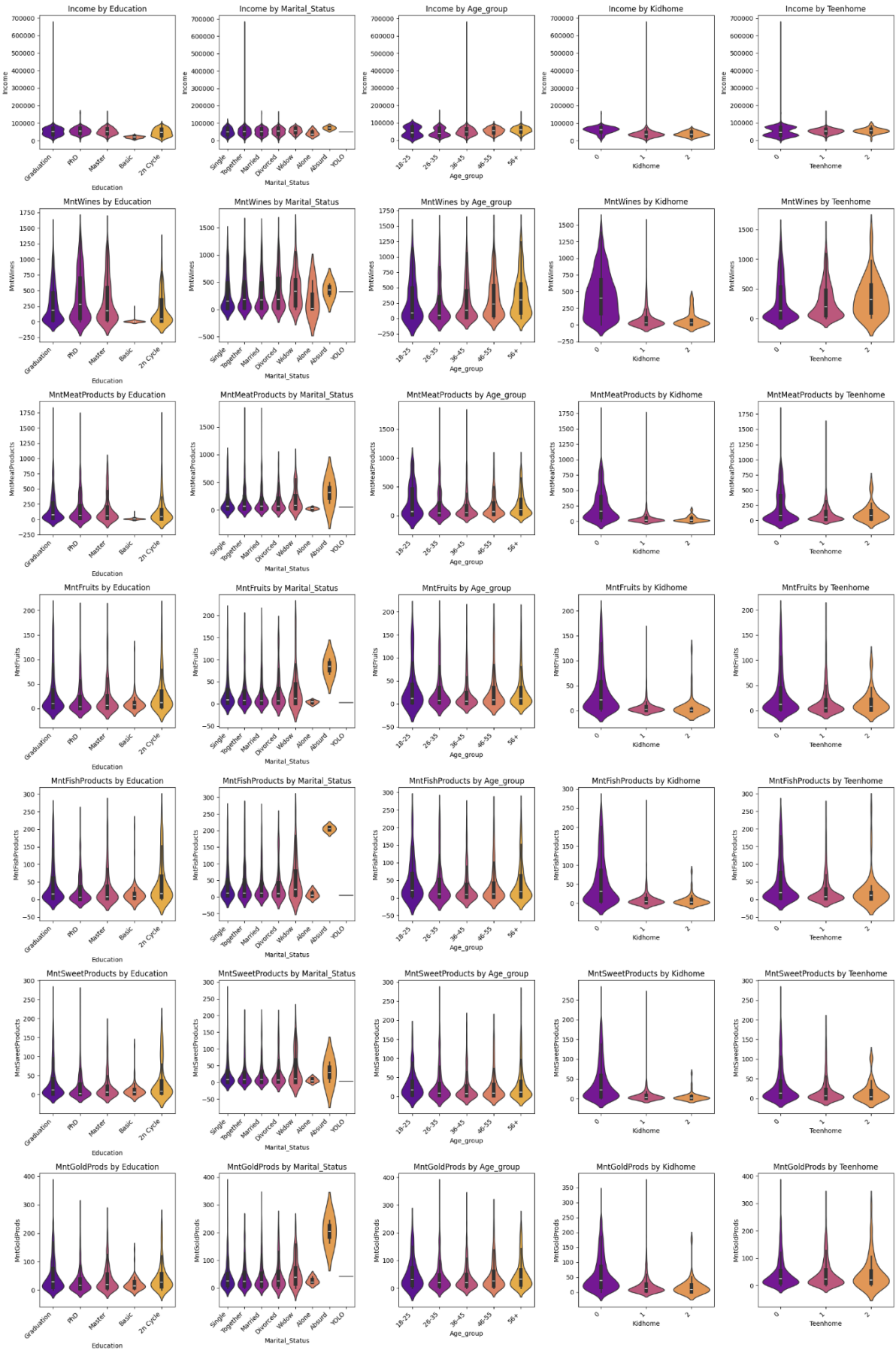
# --- Generate Box Plots ---
plt.figure(figsize=(20, 30))
n_rows_box = len(selected_numerical_cols)
n_cols_box = len(selected_categorical_cols)

plot_index = 1
for num_col in selected_numerical_cols:
    for cat_col in selected_categorical_cols:
        plt.subplot(n_rows_box, n_cols_box, plot_index)
        sns.boxplot(x=cat_col, y=num_col, data=df, palette='viridis')
        plt.title(f'{num_col} by {cat_col}')
        plt.xlabel(cat_col)
        plt.ylabel(num_col)
        plt.xticks(rotation=45, ha='right')
        plot_index += 1
plt.tight_layout()
plt.suptitle('Box Plots of Numerical vs. Categorical Variables', y=1.02, fontsize=16)
plt.savefig('numerical_categorical_boxplots.png')
plt.show()
```




```
# --- Generate Violin Plots ---
plt.figure(figsize=(20, 30))
n_rows_violin = len(selected_numerical_cols)
n_cols_violin = len(selected_categorical_cols)

plot_index = 1
for num_col in selected_numerical_cols:
    for cat_col in selected_categorical_cols:
        plt.subplot(n_rows_violin, n_cols_violin, plot_index)
        sns.violinplot(x=cat_col, y=num_col, data=df, palette='plasma')
        plt.title(f'{num_col} by {cat_col}')
        plt.xlabel(cat_col)
        plt.ylabel(num_col)
        plt.xticks(rotation=45, ha='right')
        plot_index += 1
plt.tight_layout()
plt.suptitle('Violin Plots of Numerical vs. Categorical Variables', y=1.02, fontsize=16)
plt.savefig('numerical_categorical_violinplots.png')
plt.show()
```



Observations: Write an analysis report on performing exploratory data analysis (EDA) using Python in the context of building a fraud detection system for Retail Analytics

Exploratory Data Analysis Report: Marketing Campaign System for Retail Analytics

1. Introduction

This report summarizes the Exploratory Data Analysis (EDA) conducted on the marketing_campaign.csv dataset. The primary goal of this EDA is to understand the underlying patterns, distributions, and relationships within the data, which will serve as a foundational step for building an effective marketing campaign system for retail analytics.

2. Data Loading and Initial Inspection

- The marketing_campaign.csv file was successfully loaded into a pandas DataFrame named df.
- Initial inspection using df.head() revealed the first few rows, providing a glimpse into the dataset's structure.
- df.info() showed that the dataset contains 2240 entries and 27 columns. Data types include int64, float64, and object (strings).

3. Data Cleaning

- **Missing Values:** An assessment for missing values using df.isnull().sum() revealed 24 missing values in the Income column.
- **Imputation:** These 24 missing Income values were imputed by replacing them with the median of the Income column. This ensures data completeness without significantly altering the overall distribution of the income feature.
- **Warnings:** A FutureWarning from pandas regarding inplace=True was noted, but it does not affect the correctness of the imputation.

4. Descriptive Statistics

- df.describe() provided key statistical summaries for all numerical columns, including count, mean, standard deviation, minimum, 25th percentile, 50th percentile (median), 75th percentile, and maximum values. This gave an overview of the central tendency, dispersion, and range of each numerical feature.

5. Numerical Variable Analysis

Distributions (Histograms and Box Plots)

- Histograms were generated for all numerical columns to visualize their distributions. Many spending-related features (MntWines, MntMeatProducts, etc.) showed right-skewed distributions, indicating that most customers spend lower amounts, with a few high-spending customers. Income also exhibited a right-skewed distribution.
- Box plots were used to identify outliers in numerical variables. Columns like Income, Year_Birth, and several Mnt (amount spent) columns showed clear outliers, suggesting a need for potential outlier treatment or robust modeling techniques in future steps.

An extreme Income outlier (666k) was noted and temporarily filtered for specific visualizations to avoid squashing the plots.

Relationship with Target Variable ('Response')

- **Scatter Plots (numerical_vs_response_scatter.png):** Scatter plots of each numerical feature against the Response variable (binary: 0 or 1) were generated. These plots helped visualize potential linear or non-linear relationships. For instance, higher spending on certain products (e.g., MntWines) seemed to correlate with a higher likelihood of response.
- **Correlation Matrix (correlation_matrix.png):** A heatmap of the correlation matrix was created for all numerical variables, including Response. This showed:
 - Strong positive correlations among various spending categories (e.g., MntWines and MntMeatProducts).
 - Positive correlations between Income and several spending categories.
 - Moderate positive correlations between certain spending habits (e.g., MntWines, MntMeatProducts) and Response, indicating that customers who spend more are more likely to respond to a campaign.

6. Categorical Variable Analysis

Feature Engineering

- The Dt_Customer column was converted to datetime format.
- An Age column was derived from Year_Birth (using 2014 as a reference year).
- An Age_group column was created by categorizing Age into bins: '18-25', '26-35', '36-45', '46-55', and '56+'.

Distributions (Bar Plots and Pie Charts)

- Bar plots (categorical_bar_plots.png) and pie charts (categorical_pie_charts.png) were generated for categorical variables: Education, Marital_Status, Age_group, Response, and Complain.
 - **Education:** Showed the proportion of customers across different education levels (e.g., Graduation, PhD, Master).
 - **Marital_Status:** Highlighted the distribution of marital statuses (e.g., Married, Together, Single).
 - **Age_group:** Illustrated the age demographics of the customer base.

Relationship with Target Variable ('Response') and Chi-square Tests

- Bar plots showing Response rate by each categorical variable (Education, Marital_Status, Age_group, Kidhome, Teenhome) were generated.
- **Chi-square tests** were performed to statistically assess the independence between each categorical variable and Response:
 - **Education vs. Response:** Significant relationship ($p < 0.05$), suggesting education level influences campaign response.
 - **Marital_Status vs. Response:** Significant relationship ($p < 0.05$), indicating marital status is associated with response.

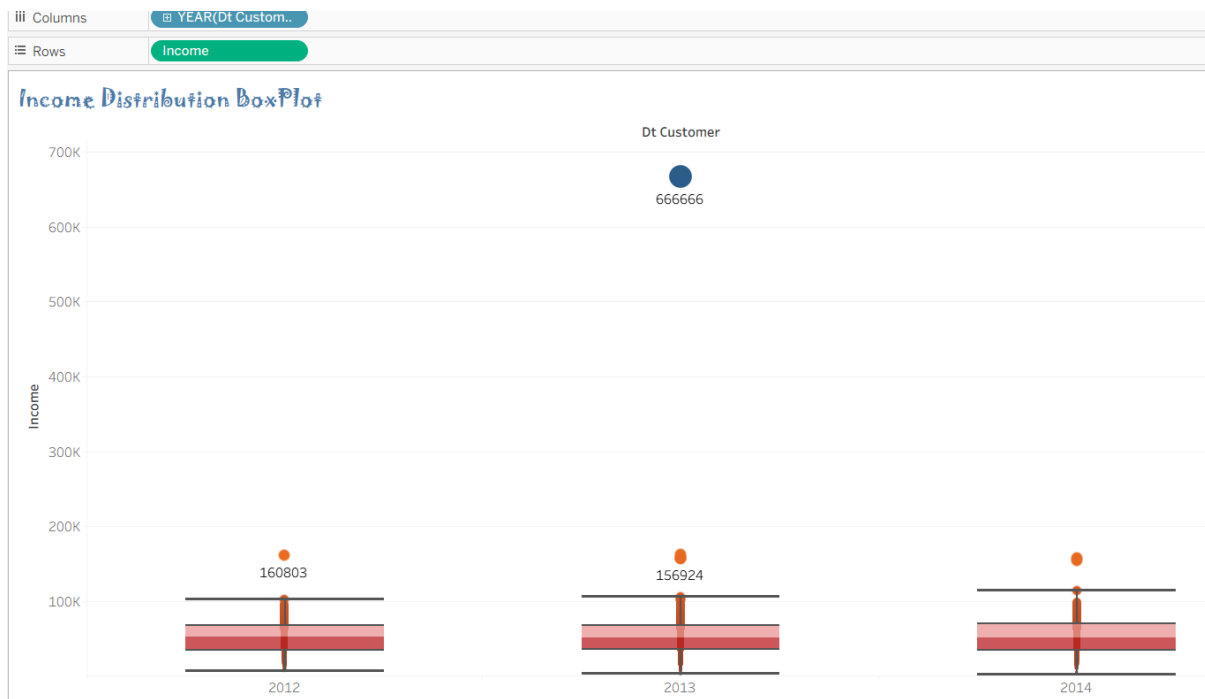
- **Age_group vs. Response:** No significant relationship ($p \geq 0.05$) found at the 0.05 significance level, implying age group alone might not be a strong predictor.
- **Kidhome vs. Response:** Significant relationship ($p < 0.05$), suggesting the number of kids at home influences response.
- **Teenhome vs. Response:** Significant relationship ($p < 0.05$), suggesting the number of teens at home influences response.

7. Numerical vs. Categorical Variable Analysis

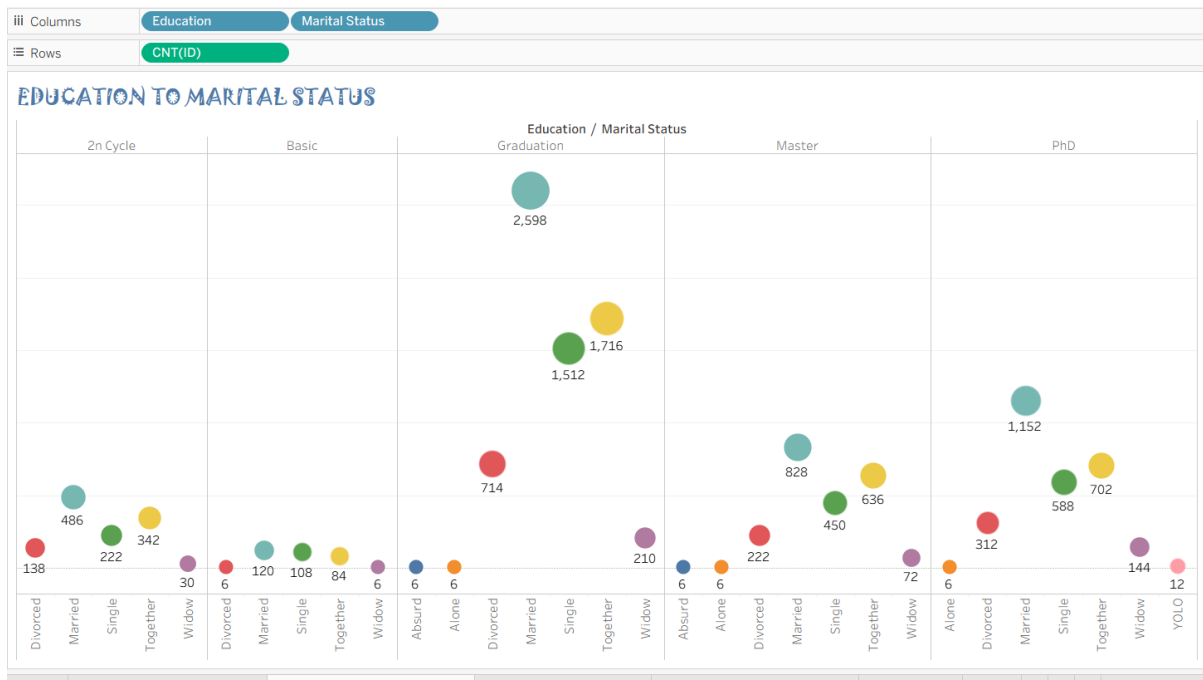
- Box plots and Violin plots were generated to examine the distributions of numerical spending variables across different categories of Education, Marital_Status, Age_group, Kidhome, and Teenhome.
 - These plots provided insights into how spending patterns (e.g., MntWines, Income) vary based on demographic factors (e.g., customers with higher education or specific marital statuses might have different income and spending profiles).
 - For instance, income and spending on various products generally tend to be higher for customers with higher education levels or those without kids/teens at home.

Tableau Tasks:

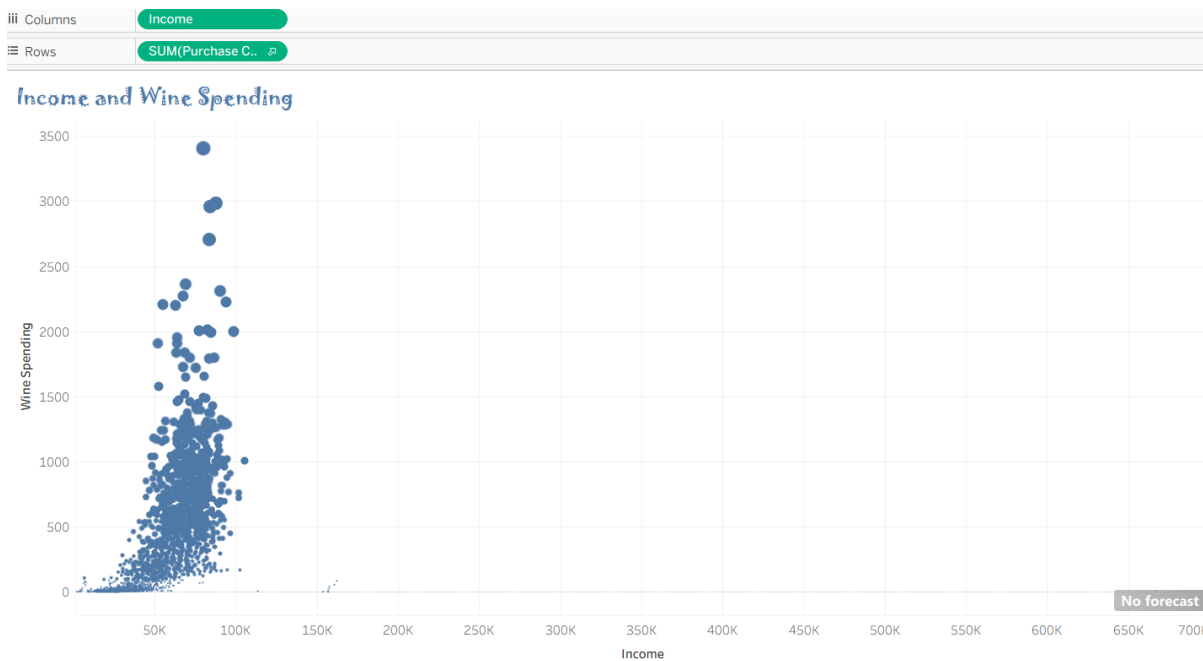
Task 1: Visualize Customer Income Distribution grouped by Year of Registration



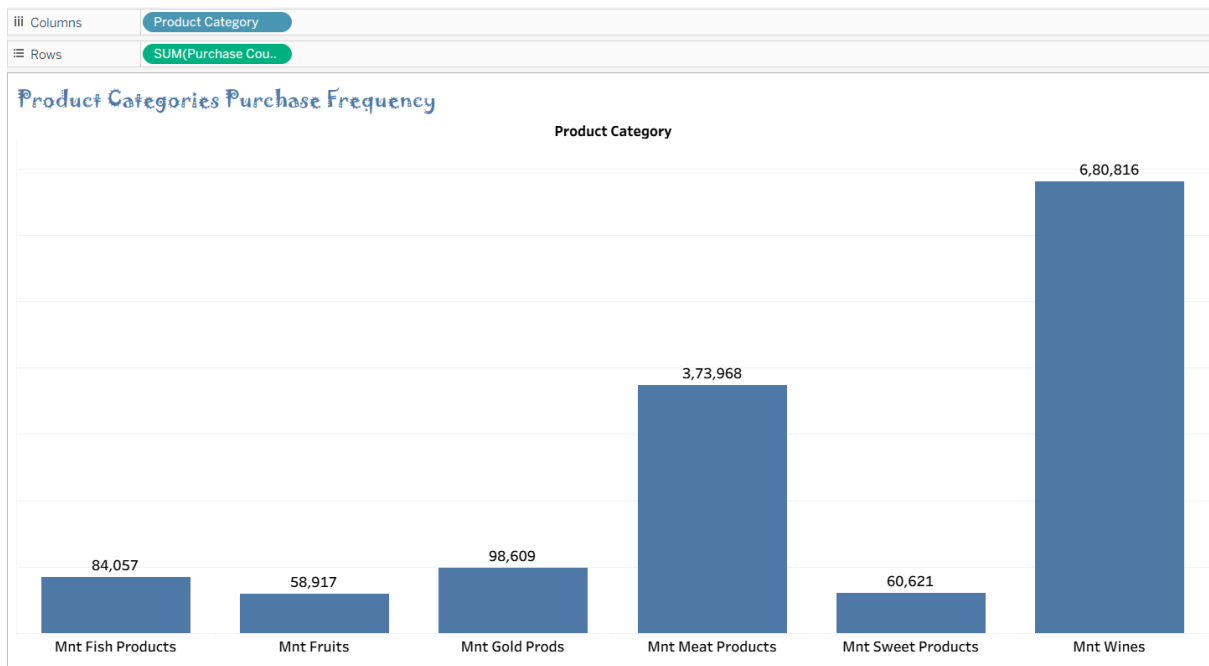
Task 2: Examine the breakdown of education levels and marital status using side-by-side circles to represent count



Task 3: Explore the relationship between Income and Wine Spending



Task 4: Analyze the frequency of purchases across various product categories



Task 5: Create a dashboard using all the visualizations

